

Progress Report

You are way behind.

Roshan Sundaram
Coherent Multicore in Verilog
March 25, 2026

What *Wasn't* Done

Despite anticipating “to generate the WARP-V cores, design the bus & caches in Verilog, and synthesize a basic version of the multicore in Vivado over spring break,” none of that actually happened, as I spent most of break fixing a 3D printer and breaking my head over ethernet cables... So, that left me with the remainder of March to work on what I intended to over break.

What Has Been Done

Bus Arbiter: For the past week-and-a-half since break, I have been focused on designing and testing a bus arbiter in Verilog. The custom module I designed interfaces with two cores and implements the logic to decide which core gets the bus. The logic consists of a round-robin arbiter that waits until a core requests the bus and, depending on which core has priority, grants the bus to a core. The finite state machine (FSM) in this module handles request, grant, and stall signals, while routing snoop requests to the opposing core during memory access.

This module was tested with testbenches in Verilog that emulated two cores connected to this central arbiter. The testbench provided signals similar to what a core would send if it wanted to claim the bus. The signals were simulated in three cases: 1) only core 0 wants the bus, 2) only core 1 wants the bus, and 3) both cores want the bus simultaneously. Examination of GTKWave traces for this testbench initially revealed some bugs in my Verilog which, upon being fixed, functioned as expected (i.e. the core that has priority gets to use the bus while the one that does not must wait for the other core to finish before it can use the bus).

Cache: Since the goal of this project is to *integrate* various Verilog modules into a coherent multicore processor, it seems like a good approach to research existing Verilog cache designs and modify them to implement the MSI coherence protocol. One such example of a cache is: <https://github.com/zebmehring/Processor-Cache>. This project looks to be a student project from a different institution a few years back, and includes various cache levels, but no coherence protocol. The goal of this week (and weekend) will be to finalize the choice of cache design and integrate it into the bus arbiter.

What Shall be Done

There is still much to do, and I admit that my progress has been stunted by two exams on 3/25 and 3/26 that I have been tunnel-visioned on. As a result, I will work extra hard this weekend and over the next few weeks to implement my multicore! With that being said, the main tasks left to complete are related to integrating the cores with the caches. Since the WARP-V cores themselves do not contain traditional data caches, I will need to add my own and make them coherent. I will then need to design a Verilog top module that integrates the bus arbiter, system-level cache, and two cores, and design extensive testbenches for them. The cores will be in RISC-V and, as a result, I will need to use a compiler (or an online tool like Compiler Explorer <https://godbolt.org>). I have not yet thought about testing or benchmarking... the task at hand is predominantly to get it *working* before anything else.